Lodos v2: Performance Tuning

Every knob that sets a simulation's throughput, what it costs, and how to set it

Lodos v2 as of 2026-11-04. A how-to guide on top of the two manuals, *Collision Detection* (`docs/collision_detection.pdf`) and *Contact Dynamics* (`docs/contact_dynamics.pdf`). Its prequel, *Creating a Simulation from an MJCF* (`docs/creating_new_sim_from_mjcf.pdf`), makes the scene. The tools are in `tools/performance_tuning/`. Built from `docs/manual/`.

About this document. A Lodos v2 scene runs correctly at its defaults, and slowly. Every buffer is sized for a worst case no real scene reaches, the solver runs its full budget, and the batch size is whatever you picked. This guide covers what sets a simulation's throughput and how to measure it, every knob, and the procedures and tools that set them. Its last part is the built-in scenes, tuned under both modes and measured before and after.

- **The knobs**: the collision capacities and tile sizes, the SDF representation and the vertex budget, MJWarp's capacities, the solver's budgets and options, the batch size.
- **The tools** in `tools/performance_tuning/`: state sets, a calibration that sizes every capacity, sweeps with an artifact guard, a stage profiler, and `tune.py`, which does all of it. The result is a tuning file that `make_sim(..., tuning=...)` reads.

Throughput is in world-steps per second: a step of one world, at the batch size stated. All numbers are measured on an RTX 4080 (16 GiB) with Warp 1.16, the step captured as one CUDA graph and replayed.

Contents

1 The short version 2

2 Measuring throughput 3

3 Where a step's time goes 3

4 State sets: what to measure from 4

5 Capacities 5

5.1 What they are, and what overflow does . . . 5

5.2 What over-sizing costs . . . 5

5.3 Measuring the demand . . . 6

5.4 Batch size and capacities . . . 7

5.5 The online alternative . . . 7

6 Lodos collision 7

6.1 Tile sizes . . . 7

6.2 Super-tiles and the compacted tile filter . . . 8

6.3 Scans, grid stride and the memory pool 8
6.4 The SDF representation 8
6.5 What was decided at bake time 8

7 MJWarp and the hybrid narrowphase **9**

8 The solver: budgets and accuracy **9**
8.1 The artifact guard 11

9 Batch size and memory **11**

10 The tuning file **11**

11 The tools **13**

12 In the live viewer **13**

13 The built-in scenes, tuned **14**

14 Knobs at a glance **19**

15 Limitations **20**

1 The short version

```
python tools/performance_tuning/tune.py my_scene.xml --out-dir tuning     # or a built-in scene's
    name
```

```
sim = make_sim(mjm, B, surfaces=surf, tuning="tuning/my_scene.tuning.json")
```

`tune.py` does what the rest of this guide explains, for one scene.

1. It explores a set of states the scene visits.
2. It times the scene as it is.
3. It picks the tile sizes.
4. It sizes every capacity to its peak over the states, times 1.5.
5. It tries cheaper solver budgets, keeping one only if it is faster and its artifacts are no worse.
6. It finds the batch size where throughput stops growing.

On the built-in scenes this made the step 1.2–29× faster under the exact mode, with a median of 4.8×, and 1.2–21× under mj, with a median of 3.2× (Section 13). Most of the gain came from the capacities and the batch size, which do not change the answer. The rest of the guide is for when you want to know why, or to do better by hand.

Table 1: Where the time goes: each stage replayed alone on one frozen state of the scene's state set, at its tuned settings and the batch it was tuned at, under the exact mode, ms per step (`profile_scene.py`, 2026-11-04). "MJWarp coll." is the profiler's `prim_collision`, "Lodos coll." its three Lodos stages.

scene, B	rigid body	MJWarp coll.	Lodos coll.	solver	integrate	step	steps/s
`arm_20_polytope`, 4096	0.424	11.891	–	38.955	0.331	50.915	80.4k
`arm_20_sdf`, 4096	0.427	–	3.061	31.514	0.334	35.085	117k
`cube_20_polytope`, 8192	0.832	0.589	–	28.761	0.644	30.128	272k
`nutbolt_1`, 2048	0.071	–	0.991	9.380	0.028	10.488	195k
`sharpa_5_sdf`, 4096	5.327	–	8.320	108.983	0.385	123.133	33.3k
`sharpa_armadillo_polytope`, 4096	0.604	1.604	–	6.183	0.088	8.576	478k
`sharpa_armadillo_sdf`, 4096	0.602	–	0.650	22.152	0.087	23.335	176k

2 Measuring throughput

- **Capture the step.** `lodos.step.make_step_graph(sim)` captures the whole step into one CUDA graph, and replaying it is how a simulation should run. Uncaptured, the Python-side launches cost more than a small scene's kernels. Every timing here is of graph replays.
- **Warm up, then time a window, from states that matter.**
  - A step's cost depends on the state: a pile in contact costs several times a pile in the air.
  - A window that is mostly a settled pile favours a solver that is fast at rest. Report the two phases apart.
  - The tools here time from a state set (Section 4): B states drawn from it, 30 steps of warm-up, then 100 steps, three times, taking the median.
- **Interleave what you compare.** Time A, B, C, then A, B, C again, not A three times and then B. The GPU's clock and temperature drift over minutes, and interleaving makes the drift fall on all of them alike (`sweep.py` does this).
- **A run that overflowed timed a different problem.** Every capacity drops work when exceeded rather than failing (Section 5): a run that dropped contacts is faster and wrong. Every timing here records every overflow flag and discards a run that set one.
- **Nothing else on the GPU.** Another process's kernels, the live viewer stepping among them, slow a timing by an unpredictable amount.

3 Where a step's time goes

A step is five passes (collision manual, *Where collision sits in a step*):

1. the rigid-body pass;
2. MJWarp's collision (`lodos.col.prim`), if the scene has MJCF geoms that pair;
3. Lodos's collision, if it has Lodos pairs;
4. the contact solve, the NLSP solver: Lodos's contacts exported into MJWarp's contact arrays, the constraint rows built, and MJWarp's solve (`lodos.cd.driver.SolverDriver.contact_dynamics`);
5. integration.

`tools/performance_tuning/profile_scene.py` times each on its own, replayed on a frozen state, and reads the funnel: how many candidates each Lodos stage leaves, against what it is allocated.

Table 2: `explore.py`'s strategies. Each start is rolled out `--horizon` steps and sampled every `--stride`.

strategy	starts
`initial`	the scene's own start, in every world (a deterministic scene: one rollout)
`scatter`	`--starts` starts drawn the way the scene draws its own (`SceneSource.sampler`: a drop's stacks, the hands' orientations), or free bodies placed at random without overlap above where they start
`actuation`	the scene's start under random control: an Ornstein–Uhlenbeck walk inside `ctrlrange`, `--sigma` of the range
`perturb`	the start and the keyframes jittered: free bodies by `--jitter` m and rad, joints by `--jitter` of their range
`keyframes`	the MJCF's keyframes as they are, with no rollout

Table 1 measures representative scenes. The pattern:

- **The solve**, with building its constraint rows, is most of a tuned step: 39 of 51 ms on the twenty armadillos as hulls, and 88–95 % of the step on the 1 mm nut-bolt, the five-hand pile and the grasp as surfaces. Its cost grows with the contacts, every one a row of three at `condim` 3 unless reduced or grouped (Section 8), and with the proximal loop's iterations. On the piles, 120 and 140 dofs, most of it is MJWarp's dense Hessian and its factorization, whose cost the dofs set.
- **The mode** moves that share. Tuned under mj, with no proximal loop, the 1 mm nut-bolt steps in 3.7 ms against the exact mode's 10.3 at 2 048 worlds (Table 6), while its collision pipeline costs no less (0.72 against 0.55 ms at 1 024 worlds, the tile search's timings): collision becomes a much larger part of its step.
- **Lodos surfaces**: under the exact mode collision is a small part of the step, 0.99 of 10.5 ms on the 1 mm nut-bolt, and within it the stage that scales with the vertex count. The 0.5 mm nut, with four times the vertices, takes 2.4 times as long to collide (Section 6.5).
- **Hulls**: MJWarp's collision, its broadphase and the hybrid narrowphase, is 12 of the twenty-armadillo pile's 51 ms. Its cost grows with the hull pairs whose boxes overlap, not with the contacts.
- **The rigid-body pass** is small: 5.3 of 123 ms on five hands, 28 dofs each.

4 State sets: what to measure from

Every capacity is sized to a demand, and the demand is a property of the states the scene visits. A state set (`lodos.utils.states`) is the list of states that every tool runs from. They all use the same states, so that the capacities, the timings and the accuracy they report are about the same thing.

```
from lodos.utils import states
S = states.from_keyframes(mjm)                        # the MJCF's keyframes
S = states.from_qpos(mjm, qpos, qvel, ctrl=u)         # any (N, nq) arrays: a controller's trace,
    RL rollouts
S.save("scene.states.npz"); S = states.load("scene.states.npz", mjm)   # refuses another model's
```

Exploration. `tools/performance_tuning/explore.py` makes a set from the scene alone. It rolls each of its starts out for a horizon and samples the state every few steps (Table 2). A still frame misses the impacts: Lodos-v1's MPC (`papers/lodos_v1.pdf`) sized its `njmax` on 10 ms frames of a 2 ms step and got 211 rows where the run needed 398. Filters drop states that went non-finite and states deeper in penetration than `--max-depth`. The report says whether the set's heaviest state is still being found: the running peak of contacts per world against the number of states, in a random order. If the last rise is near the end, explore more.

Layouts. Some capacities are *batch totals*, shared by all B worlds, and some are *per world* (Table 3). A batch total depends on how the B worlds of a batch are assigned to states:

- **independent** (the default): each world its own state. This is reinforcement learning's many independent episodes. A batch's totals are about B times the *mean* load, and the larger the batch the closer.
- **shared**: every world of a batch from one state, each with its own controls. This is model-predictive control's one plant and many candidate futures. A batch's totals are about B times *that state's* load, so the heaviest state sets them, at any B.
- **explicit**: an index per world.

A tuning file records the layout it was measured under. A file measured independent and run shared will overflow on the first heavy state; measure the way you will run.

Controls. During a rollout the actuators can:

- hold each state's control (`hold`, the default);
- be zero (`none`);
- walk at random inside `ctrlrange` (`random`);
- play the set's own control sequences, `(N, H, nu)` (`sequence`).

An MPC's state set should carry its candidate control sequences: the controls are what take the plant into its heavy states.

5 Capacities

5.1 What they are, and what overflow does

Every array of a `Sim` is allocated once, at construction, because a captured graph needs stable addresses. Every count that varies is bounded by a capacity chosen then (Table 3). A capacity that is exceeded **drops** work rather than failing:

- a Lodos stage keeps its first survivors;
- MJWarp drops contacts past `naconmax`, broadphase pairs past the same `naconmax`, and rows past `njmax`;
- Lodos's contacts, and contact groups, are exported into MJWarp's contact arrays after MJWarp's own, and the ones past `naconmax` are dropped the same way;
- the hybrid drops EPA pairs and manifold contacts.

Each leaves a trace, a sticky flag or a count past its capacity, that nothing in the step reads. A run that overflowed simulated a different problem, faster.

5.2 What over-sizing costs

It depends on the capacity.

- **Lodos's collision capacities** cost launch width. Every compaction's scan reads its length from the device (`dyn_scan`), and stage 1 walks a hardware-sized grid (`grid_stride`), so an idle slot is nearly free, but not free. Measured on the SHARPA hand at $B = 8192$, each knob alone (collision manual, *Capacities and overflow*):
  - 3× over-sized costs 2.2 % of the step for `capacity1`, 1.2 % for `capacity2` and 5.2 % for `capacity_tile`;
  - each further doubling costs 1.3, 1.5 and 6.7 %.

Table 3: Every capacity of a `Sim`: what it bounds, whether it is a batch total or per world, and where it is set. "Per world" in MJWarp's names (`nconmax`, `nccdmax`) means times B and shared: what a calibration gives them is a share of a batch total.

capacity	bounds	kind	set by
`capacity_tile`	the surviving tiles, and with them stage 1's candidates	batch	`col_kwargs`
`capacity2`	the voxel-SDF survivors and the contacts (`nCa3_max` = `nCa2_max`)	batch	`col_kwargs`
`capacity1`	the tile filter's compacted launch; 1.0 launches densely and cannot overflow	batch	`col_kwargs`
`naconmax`	MJWarp's contacts, Lodos's exported ones and the contact groups among them, and its broadphase pairs	batch	`make_sim`
`nconmax`	`naconmax` / B when `naconmax` is not given	share	`make_sim`
`nccdmax`	the hybrid's EPA and manifold scratch, times B	share	`make_sim`
`njmax`	MJWarp's constraint rows: every contact's, joint limits and friction loss	world	`make_sim`
`njmax_nnz`	a sparse Jacobian's nonzeros (models over 32 dofs)	world	`make_sim`; derived from `njmax` by default

The defaults of 1.0 are worse than 3× over on a real scene: the five-armadillo drop's worst-case buffers are about 4 GB at $B = 4096$.

- **MJWarp's capacities** cost much more, because several of its kernels launch over `njmax` rows and `naconmax` contacts. On the five-armadillo drop, the scene's own capacities (sized for the live viewer's one world: 4 096 contacts and 12 288 rows per world) against calibrated ones made the step 3.4× faster at the same batch size, with identical contacts.
- **The row capacity also sizes two of the solver's options.** `fused_newton` keeps a world's rows in shared memory and refuses an `njmax` whose rows do not fit there, and `row_space`'s memory grows with `njmax`: a world's rows times its dofs, and its rows squared, up to 1 024 rows. Both want a calibrated `njmax` (Section 8).

5.3 Measuring the demand

```
python tools/performance_tuning/calibrate.py arm_5_sdf --states arm_5_sdf.states.npz --B 4096 \
    [--layout independent|shared] [--horizon 200] [--controls hold|none|random|sequence] \
    [--safety 1.5] [--out arm_5_sdf.tuning.json] [--update]
```

`lodos.utils.calibrate.measure` rolls every batch of a state set out for `--horizon` steps. After every step, inside the captured graph, a `Tracker` keeps the running maximum of every count on the device:

- Lodos's stage counts (the stages' own `ct_peak`), and its contacts per world;
- MJWarp's contacts (Lodos's exported ones among them) and broadphase pairs, its rows per world, and its contacts per world;
- the hybrid's EPA, manifold and contact counts.

Every count is read raw, before any clamping, so an overflowed stage still reports its true demand. The stages behind it saw fewer than they would have, though. So `lodos.utils.calibrate.calibrate` grows whatever overflowed and measures again, until nothing does. Only then are the peaks the demand. The capacities are then `--safety` times the peaks (`lodos.utils.calibrate.settings`).

To start that loop somewhere sensible, `calibrate.py` first measures a small batch, 64 worlds at the scene's

own capacities, which hold at that size. It starts the real batch from those fractions: a smaller batch's mean is noisier, so its peak fraction is the larger.

> **The counters that are easy to miss**. **MJWarp's broadphase pairs** are written into arrays of the same `naconmax` as its contacts, and counted separately (`d.ncollision`). A scene of many hulls has many more overlapping boxes than contacts. On the SHARPA grasp's 121 hulls, sizing `naconmax` by contacts alone dropped broadphase pairs, missed contacts, and threw the armadillo out of the hand in a tenth of the worlds.
> **The hybrid's EPA and manifold scratch**, sized by `nccdmax` × B. `lodos.utils.calibrate.peaks`, which measures one rollout rather than a state set, reads the broadphase's pairs too, every step by default (`every=1`), and `lodos.utils.calibrate.suggest` sizes `nconmax` to hold the larger of the two peaks; neither measures the hybrid's counts.

5.4 Batch size and capacities

A batch total measured at one B is not right at another.

- **Fractions** (`capacity_tile`, `capacity2`, `capacity1`) already scale with B. Under the independent layout they are conservative at a larger batch, which is closer to its mean, and unsafe at a smaller one.
- **Absolute totals** (`naconmax`) must be scaled by the ratio of the batch sizes.

`lodos.utils.tuning.apply` does both:

- at the measured batch size or above, it takes the file's settings whole and scales the absolute totals;
- below it, it leaves the batch totals out, with a warning.

The live viewer runs one world, and takes a tuning file without its capacities (Section 12). Settings measured at 8192 worlds and applied whole at 2048 overflowed: `tune.py`'s own final check caught it.

5.5 The online alternative

`lodos.col.sdf.collide.Autosizer` sizes Lodos's collision capacities on a live run instead: it shrinks onto the peaks it sees and grows a stage that overflows. It needs no state set and converges lower than an offline calibration (0.047 against 0.073 on the SHARPA hand), because it sizes to the batch's mean behaviour. But it sizes to what it has seen: a capacity shrunk in a quiet phase is overrun by a busy one, for up to `every` steps, and those contacts are lost. It also resizes in place, which invalidates every captured graph. Where a dropped contact matters, run it once as an instrument and build the real context from what it found (`suggest_capacity`). It sizes only Lodos's collision, not MJWarp's.

6 Lodos collision

6.1 Tile sizes

`tile_size` (default 8) is how many consecutive Hilbert-ordered vertices a tile holds. A tile is rejected whole against the other geometry's voxel SDF, so the tile size trades the tile level's own cost (small tiles) against how many vertices survive into stage 1 (large tiles). The collision manual's *Tiles* has the curves: on the SHARPA hand, 131.6 ns per world at 4, 106.1 at 8, 216.1 at 128.

- **Per directional pair.** `tile_size={None: 8, (1, 0): 16}` sizes the segments whose vertices come from geometry 1 and whose field from geometry 0, and gives the rest 8. A 55-tile palm and a 6-tile phalanx want different sizes. Sizes that differ cost 4.9 ns per world: a slot's tile is then a lookup, not a division (`partition`, which `make_collision_context` turns on by itself exactly then).

- **Small geometries**: a geometry of fewer than 2·`tile_size` vertices is one or two tiles that cull nothing. Lower its pairs' size, down to 1, where a tile is a vertex.
- `tile_tuner.py` groups the segments by their (vertex surface, SDF surface) assets. It searches 4, 8, 16 and 32 for the largest groups, one at a time, timing the collision pipeline alone on a frozen batch, each candidate at 1.5× its own demand. On the built-in scenes, under the exact mode, it kept 8 for the five-armadillo drop the default was chosen on, the nut-bolt and the grasps. It moved to 16 or 32 on the large piles, the hands' minor link pairs, most of the NIST nuts against their bolts and some bodies against the floor, and took up to 44 % off the collision step there (Section 13).

6.2 Super-tiles and the compacted tile filter

`super_size` groups tiles into super-tiles, tested like tiles, whose rejection removes their children from the tile filter's launch. It pays for fine tiles. It needs the compacted launch (`capacity1` < 1), is off by default and is per directional pair, like `tile_size`. Keep it off for box-like geometry, where this stage has twice measured slower.

`capacity1` < 1 compacts the box filter's surviving segments first, so the tile filter runs over the tiles of active segments only. On the SHARPA hand the box test rejects 90 % of the segments, and the tile filter's grid shrinks five- to sevenfold. At 1.0 it launches densely, bit-identically. `tune.py` measures both and keeps the faster.

6.3 Scans, grid stride and the memory pool

`dyn_scan` and `grid_stride` (both on by default) are what make an over-sized capacity cheap. Leave them on; off, each compaction scans its whole arena. `mempool_release_threshold` (1 GiB) lets Warp's memory pool keep the CUB scans' scratch: Warp's default of 0 makes each scan's temporary allocation a driver round trip, about 0.6 ms a scan.

6.4 The SDF representation

A bake can be evaluated as a tricubic B-spline (S_{c2}, `"cubic"`, the default, 64 coefficients per query) or as trilinear interpolation of the same knot samples (S_{c0}, `"linear"`, 8 samples per query). The choice is made when the surface is loaded, and the narrowphase kernel is compiled for it.

- S_{c0} makes the narrowphase cheaper.
- Its normals are only C^0: a contact sliding across a voxel face sees its normal jump.

The nut-bolt ships both (`nutbolt_*` and `nutbolt_*_trilinear`); their tuned throughputs are in Table 6. There S_{c0} cut the collision pipeline's time by 30–44 %; the tuned step was no faster for it under the exact mode, and 13–16 % faster under mj (Section 13). Choose S_{c0} when the narrowphase is a large share of the step and the contacts do not slide far.

6.5 What was decided at bake time

- **The vertex budget is the largest lever on Lodos's collision**, and without contact reduction on the solve too.
  - Every vertex of a vertex side is a candidate in every pair it is in, before the filters, so stage 1's arena and the tile filter's grid are proportional to it.
  - Every penetrating vertex is a contact, and every contact a row of three in the solve, unless the reduction (Section 7) or contact groups (Section 8) stand in for it.

  - The 0.5 mm nut has 4.0 times the 1 mm nut's vertices (19 100 against 4 799), and its collision pipeline takes 2.4 times as long (1.29 against 0.55 ms a step at 1 024 worlds, the tile search's timings in `tools/performance_tuning/tuned/`). Its tuned step is only 1.12× slower (177k against 199k world-steps/s at 2 048 worlds): the solve is most of it, and the reduction, on for both nuts, hands it at most 240 contacts a segment.
  - Lowering a body's target in step 3 of the mesh pipeline is the first thing to try on a surface scene whose collision is slow.
  - A tuning file records each surface's vertex count and its pipeline records, and goes stale when they change (Section 10).
- **The voxel grid** costs memory, 4 bytes per voxel and per coefficient: 3–5 MB a surface on the SHARPA hand. It costs no time per step beyond cache behaviour: each query reads its own 8 or 64 values. A finer grid makes the voxel prefilter reject more tightly.
- **Which side supplies the vertices**, and **which pairs are declared**, are speed choices too. A pair costs its box test at every step, and its vertex side's tiles whenever the boxes overlap. Declare both directions only where the second finds contacts the first misses.
- **Inner spheres** add primitives to a geometry. A scene where no geometry has any runs the sphere-free kernels, compiled for it.

7 MJWarp and the hybrid narrowphase

- **The hybrid** (`make_sim(hybrid=ALL_TYPES)`, the default) takes box–box, box–mesh, mesh–mesh and plane–mesh pairs. Against a float64 ground truth it misses no contact where MJWarp's own narrowphase misses 27–250 per scan, and its narrowphase is 7.6–18.5× MJWarp's on twenty armadillos (collision manual, *Exactness and speed*). There is no reason to turn it off except a model it refuses.
- **Hull counts** set the hybrid's cost: hull pairs whose boxes overlap go to GJK, and those in contact to EPA and the manifold. A coarser decomposition (a higher CoACD threshold in step 2) is the polytope scenes' vertex budget.
- **Contact reduction** (`make_sim(contact_reduction=True)`) hands MJWarp at most 240 of each Lodos pair segment's contacts, chosen deterministically (Newton's reduction). Every contact is a row of three in the solve, so this is the solver's largest lever on a dense contact patch. It made the exact mode's proximal solve 18.5× faster on the nut-bolt (at slop 0, measured on 2026-10-24); the nut-bolts and the NIST scenes ship with it on. Grouped pairs are reduced too: their groups are made of the contacts it keeps. It changes the answer, so it is guarded like a budget (Section 8).
- **The sparse Jacobian.** A model over 32 dofs gets a sparse constraint Jacobian, whose nonzeros `njmax_nnz` bounds. By default it is estimated from `njmax` and the widest contact, Lodos's pairs counted beside the MJCF's own (`lodos.mjcf.io`), so sizing `njmax` sizes it too. When it was too small, a pile of twenty armadillos fell through the floor without a word.
- **The EPA scratch** is `nccdmax` × B rows of polytope workspace. Left out, it is sized to the whole contact capacity: 2.5 GiB for twenty armadillos at $B = 1024$.

8 The solver: budgets and accuracy

The capacities and the tile sizes do not change the answer. With no overflow, the contacts are identical. The budgets do: fewer iterations are a less converged solve. Every budget change here is held to the scene's own settings by the artifact guard (Section 8.1).

Table 4 lists them. `tune.py` tries two of them: 5 and 3 proximal iterations, and the contact reduction on a scene with Lodos surfaces that does not ship with it on; under mj, which runs no proximal loop, only the reduction (Section 11). The others are for `sweep.py` and a scene's own check. Two were measured on

Table 4: The solver's throughput knobs, its options (`solver_kwargs`) unless named otherwise. "Answer" says whether a knob changes the trajectory; "same" is to rounding.

knob	what it buys, what it costs	answer
`prox_iters_per_qp` (10)	the exact mode's proximal loop, its outer iterations: fewer is less converged	changes
`prox_tol_z`, `prox_tol_sigma` (`1e-3` each)	the loop's stopping tolerances, on the anchor and on the de Saxcé term	changes
`contact_reduction` (`make_sim`)	at most 240 contacts per Lodos pair segment	changes
MJWarp's `opt.iterations`, `opt.tolerance`	the Newton solve's cap and tolerance, from the MJCF (100 iterations in the built-in scenes')	changes
`fused_newton` (off)	the Newton solve, and the exact mode's proximal rounds, in one kernel per world, for small models (up to 16 dofs) at a calibrated `njmax`: the exact step 2.3× faster on the 1 mm nut-bolt	same
`groups`, `normal_bins`, `moment_bins` (off; 1, 1)	a pair segment's contacts in each normal bin and moment sector solved as one contact, a ray (one row) or a cone (three): fewer rows, another problem	changes
`row_space` (off)	MJWarp's Newton system solved through the rows rather than as the dofs' dense Hessian: pays where a world has fewer rows than dofs, as with ray groups on the piles	same
`shared_hessian` (from 64 padded dofs)	MJWarp's sparse Hessian (more than 32 dofs, up to 144 padded) summed per world in shared memory rather than rebuilt in global memory every iteration: the same Hessian, so the same iterates; on by default wherever it can be built on a model of 64 padded dofs or more (48 dofs; below, it measured slower); its block, 512 threads a world, is a GPU-specific tuning parameter, chosen on an RTX 4080 (*Contact Dynamics*)	same

2026-10-29:

- **`fused_newton`** solves the same problem to the same iterates. On the 1 mm nut-bolt at 2 048 worlds it took the exact step from 194k to 448k world-steps/s, and it passed the artifact guard, its metrics equal to the third digit. It refuses a model of more than 16 dofs, and a row capacity whose rows do not fit in shared memory: a scene's own, sized for the viewer's one world, is usually too large (the nut-bolt's is 24 576 rows a world).
- **Ray groups with `row_space`** cut the twenty walled armadillos' rows from 125 to 31 a world and ran their exact step 3.1× faster than per contact without the shared Hessian, at 4 096 worlds (2.6× on five walled hands). Cone groups, 94 rows a world there, gained nothing: their row system is as large as the Hessian. Groups change the problem: on the contact dynamics manual's slope test, ray groups let a cube creep 53 mm in a second where per contact and cone groups hold it, and one group a pair cannot hold the NIST nut on its thread.

Which mode to run is the larger choice. The mj mode solves MuJoCo's soft problem with no proximal loop: tuned, it runs a median 2.1 times the exact mode's throughput on the built-in scenes (2.8 on the 1 mm nut-bolt), though slower on 7 of the 67, and its contacts creep where the exact mode's hold, 1.0 mm/s on the same slope. The contact dynamics manual weighs the two; Table 6 measures both modes on every built-in scene, tuned.

8.1 The artifact guard

A budget is kept only if it is faster by 3 % *and* `common.no_worse` passes against the scene's own settings, on the same states, at 512 worlds (the guard's metrics are means and percentiles over worlds, and 512 measure them as well as thousands):

- no new non-finite or escaped worlds;
- no capacity overflow during the guard's rollouts;
- the penetration within 1.25× + 0.2 mm, both its mean and its 99th percentile, per world, sampled every 20 steps over 400;
- the creep at rest within 1.5× + 0.2 mm/s, and the speed at rest within 2× + 0.02, over 500 steps from the set's settled states, which take up to half the batch;
- for the scenes that are not chaotic (the incline, the backspin ball, Painlevé's box), every body's path within 5 mm of the baseline's.

Its noise floor. Two runs of the same physics do not land in the same place. The capacities change only the order of floating-point sums, and MJWarp's solve sums with atomics, whose order varies from run to run. On a chaotic scene that is enough to move the metrics: the SHARPA grasp's speed at rest varied twofold.

So `tune.py` measures the gap between the baseline's artifacts and the capacities-only settings' on the same states, which is each metric's run-to-run spread. It widens each limit by twice that gap, and the path limit by twice the paths' own spread. A budget that passes one rollout and fails the final check is dropped, and the settings are recalibrated without it.

Its thresholds are heuristics. They catch what was seen: the grasp thrown, creep doubled, penetration up by half. A scene with a quantity of its own to hold (a seat's depth, a grasp's slip) deserves a check of its own beside them.

9 Batch size and memory

Throughput grows with the batch size until the GPU is busy, then flattens. Memory grows linearly, and the largest batch that fits is set by the capacities. `tune.py` doubles the batch from the family's starting size while throughput grows by more than 5 %, recalibrating at each size, and keeps the fastest. Each tuning file records the batch it was measured at (Table 6). The light scenes saturate later than the heavy ones. Under the exact mode the single bodies of the exactness scenes (the incline, the backspin ball, Painlevé's box) were fastest at 8 192–32 768 worlds (under mj up to 65 536). Of the other 61 scenes, 45 were fastest at 2 048–4 096 and 12 at 8 192: the five SHARPA hands as hulls, the twenty cubes as boxes, the Shadow grasp, the driven nuts and five of the nine NIST nut-bolts; the twenty-hand piles at 1 024.

10 The tuning file

`lodos.utils.tuning` holds a scene's settings in one JSON file per scene (format `lodos-tuning/2`). The settings are four groups at the top level, the ones `make_sim` takes: `collision`, `runtime` (the capacities of the buffers both pipelines' contacts share: `naconmax`, `nconmax`, `njmax`, `nccdmax`, `njmax_nnz`), `sim` (`contact_reduction`) and `solver_kwargs` (the budgets the tuning changed, and the mode of a file tuned under mj). Beside them are `B`, the batch they were measured at, the `peaks` the capacities came from, the `throughput` and `artifacts` they reached, the `baseline` they were compared with, and the tile search's record:

```
{ "format": "lodos-tuning/2", "scene": "arm_5_sdf", "B": 4096, "layout": "independent",
  "fingerprint": {
    "model": {...}, "geometry": "...", "timestep": 0.002, "hulls": { "hulls": 0 },
    "surfaces": [ { "kind": "surface", "vertices": 451, "faces": 898,
                    "grid_N": [85, 76, 100], "voxel_h": [...], "spline_type": "cubic",
                    "spheres": 0, "extent": [...], "sha": "079adb9906bc5b25",
                    "file": "viewer/assets/surfaces/armadillo/surface_data.npz" }, ... ],
    "pairs": 25 },
  "states": { "path": ".../arm_5_sdf.states.npz",
              "digest": "8ccc73cd2032f603", "size": 2560, "horizon": 200,
              "controls": { "kind": "hold" }, "source": {...} },
  "safety": 1.5, "created": "2026-11-01T02:50:20", "commit": null,
  "device": "NVIDIA GeForce RTX 4080",
  "collision": { "capacity_tile": 0.134, "capacity2": 0.0259 },
  "runtime": { "naconmax": 51098, "nconmax": 13, "njmax": 77 },
  "sim": {}, "solver_kwargs": {},
  "peaks": {...}, "throughput": { "world_steps_per_s": 308526, "B": 4096 },
  "artifacts": { "tuned": {...}, "baseline": {...}, "ok": true, "why": "" },
  "baseline": { "B": 2048, "settings": { "runtime": { "nconmax": 4096, "njmax": 12288 }, ... },
                "throughput": {...}, "artifacts": {...}, "guard_B": 512 },
  "tiles": { "groups": {...}, "trials": [...], "base_ms": 0.163, "best_ms": 0.163,
             "sizes": {...} } }
```


- **Applying it.** `make_sim(..., tuning=path)`, or `SceneSource.make_sim(..., B=, tuning=)` for a viewer scene, fills whatever the call leaves out (`lodos.utils.tuning.apply`, `merge_into_make_sim`):
  - MJWarp's capacities and `contact_reduction`;
  - the solver's options the file names;
  - the collision options that `surfaces.col_kwargs` does not name.

  Arguments given explicitly win. A scene's own defaults (a `SceneSource`'s `caps`, `sim_kwargs`, `solver_defaults` and `mode_defaults`) give way to the file. Batch sizes follow Section 5.4. `lodos.utils.tuning.describe(file)` summarizes a file in two lines.
- **Staleness.** The fingerprint covers:
  - the model's structure, geometry and timestep;
  - every hull set's counts and hash;
  - every Lodos surface's vertex and face counts, SDF grid, voxel size, spline type, inner spheres and a hash of its arrays;
  - when they sit beside its bake, the mesh pipeline's records of how it was made: decomposition threshold, vertex target, bake settings.

  `lodos.utils.tuning.stale(file, mjm, surfaces)` lists what changed. `make_sim` warns on a stale file, and `calibrate.py --update` re-measures it.
- **The records** answer "how was this surface made?" from the tuning file alone. That matters, because the vertex budget is the largest lever on collision (Section 6.5). `tools/performance_tuning/tuned/sharpa_armadillo_sdf.tuning.json` carries each link's CoACD threshold and hull count, its Instant Meshes target and its bake's grid. A surface with no records beside its bake, like the armadillo above, has none.

Table 5: `tools/performance_tuning/`: a scene is a built-in scene's name or an MJCF file the viewer can load.

script	does
`explore.py`	a state set: strategies, filters, the running-peak report (Section 4)
`calibrate.py`	every capacity at a batch size, into the tuning file (Section 5.3); in code, `calibrate.calibrate_section`
`tile_tuner.py`	the tile size of each kind of segment (Section 6.1); `--write` into the file
`sweep.py`	any knobs over a grid, interleaved rounds, optionally guarded and recalibrated per configuration
`profile_scene.py`	each stage's time and the funnel (Section 3)
`tune.py`	all of it for one scene: baseline, tiles, capacities, compaction, guarded budgets, batch size; `--copy-from` makes a variant that differs from another scene only in its joint rows follow that scene's tiles, budgets and batch size; `--keep-tiles` takes the scene's file's tile sizes rather than searching them again (a missing, tile-less or stale file is refused); `--mode mj` tunes under a mode other than the scene's own, from a state set explored under it, into `<out-dir>/mj/`, the file recording the mode
`tune_all.py`	`tune.py` over every built-in scene, a process each; `--mode mj` under mj
`summarize.py`	the table of every tuned scene, before and after: `tuned/README.md`
`time_variant.py`	a variant of a scene's tuned settings with other solver options (by default the mj mode without the proximal loop, labelled `lodos_mj`, for the comparison with Newton's solvers), recalibrated at each batch and timed as `tune.py` does, guarded; into a file a scene in `--out`, not the tuning file

11 The tools

Table 5 lists them. `common.py` is what they share: scenes by name, builds from a set of settings, `throughput`, `artifacts`, `no_worse`. A knob of `sweep.py` is `<group>.<name>` in one of the tuning file's four groups. Examples:

```
python tools/performance_tuning/sweep.py nutbolt_1 --B 4096 \
    --tuning tools/performance_tuning/tuned/nutbolt_1.tuning.json \
    --knob solver_kwargs.prox_iters_per_qp=10,5,3 --knob sim.contact_reduction=true,false \
    --guard --recalibrate --rounds 3
python tools/performance_tuning/sweep.py nutbolt_1 --B 2048 \
    --tuning tools/performance_tuning/tuned/nutbolt_1.tuning.json \
    --knob solver_kwargs.fused_newton=false,true --guard
python tools/performance_tuning/profile_scene.py sharpa_5_sdf --B 4096 \
    --tuning tools/performance_tuning/tuned/sharpa_5_sdf.tuning.json --states sharpa_5_sdf.states.
    npz
```

The profile runs at the file's own batch size: below it, the file's batch totals give way to the scene's own (Section 5.4), which are sized for the viewer's one world.

12 In the live viewer

The viewer's **Performance** panel (`viewer.performance`) does five things.

- **The tuning file.** It loads the scene's file, by default `tools/performance_tuning/tuned/<scene>.tuning.json`, the exact mode's (a file tuned under mj loads by its path), shows its settings and the throughput before and after, and flags it stale. *Apply knobs* takes the file's tile sizes, which rebuild the runners, and its solver budgets and contact reduction, which become Lodos's options in the Sim panel,

where they can be seen and changed. It does not take the capacities: the viewer runs one world and grows its own capacities when a step overflows them. A tuning file tunes Lodos, not Newton's sims (MJWarp, Kamino), and is not applied under them. With its checkbox, or `python -m viewer --tuned`, the file is applied on every scene load and sim switch.

- **The state set.** *Add frame* adds the state on screen to the scene's set, and *Add take* adds a take every few frames. Save and load the set; select a state in the list to show it.
- **The tools**, in the background at a chosen batch size: explore, calibrate, profile, or the whole tuning. A finished calibration or tuning reloads the file, and a profile shows its stage table. They time on the GPU the viewer is using, so pause the viewer for clean numbers.
- **Counters.** The state on screen against the viewer's own capacities: each Lodos stage (candidates, tiles, voxel survivors, contacts) and MJWarp's contacts and rows (*Runtime contacts*, *Runtime rows*); under one of Newton's sims, its contacts.
- **Tile size**, for every pair at once, with a rebuild.

13 The built-in scenes, tuned

`tune_all.py` tuned the 71 built-in viewer scenes on 2026-11-01 on an RTX 4080, each under the exact mode and, with `--mode mj`, under mj, every scene on the elliptic cone. 67 of them have a file under each mode: the four twenty-hand piles as surfaces ran out of memory exploring their states. The files are in `tools/performance_tuning/tuned/` and `tuned/mj/`, each with its baseline and every trial in `<scene>.report.json`. Table 6 is the result.

A file tuned under mj names the mode in its `solver_kwargs`, so a build from it runs mj, and its capacities hold that mode's demand: MuJoCo's soft contacts let bodies sink further, and the walled five-hand pile as surfaces peaks at 874 contacts and 2 750 rows in one world under mj, against 158 and 603 under exact. The files were measured with the shared Hessian on wherever MJWarp could build it, from 33 dofs; by default it is on from 64 padded dofs (Table 4), so `arm_box_6` (36 dofs) and the Shadow grasp (34) run without it unless asked (`shared_hessian=True`).

- **Baseline** is the scene's own settings: its own MJWarp capacities, which the viewer sized for one world, the collision defaults (every fraction 1.0, tile size 8) and the solver defaults. It is measured at the family's batch size, or the largest that fit.
- **Tuned** is the file, at the batch size the sweep found fastest.

Over all scenes, the tuned settings against the baseline: 1.18–29.27× (median 4.85, 67 scenes). Changes kept: compact in 1, prox 3 in 23, prox 5 in 15, red in 6, tiles in 23 scenes. 67 of 67 passed the final artifact check.

Table 6: Every built-in scene, before and after tuning: batch size, world-steps per second and the ratio; what the tuning changed besides the capacities ("prox" *n*: `prox_iters_per_qp`; "red": `contact_reduction`; "tiles": tile sizes per pair; "compact": the compacted tile filter) and the final artifact check (✓ passed); "(mj)": a file tuned under mj (`tune.py --mode mj`).

scene	baseline		tuned			
	B	steps/s	*B*	steps/s	ratio	changed, check
`incline_sdf`	2048	86.4k	8192	1.81M	20.92	prox 3, red; ✓
`incline_sdf (mj)`	4096	467k	65536	5.87M	12.56	tiles; **no**
`incline_polytope`	2048	458k	16384	4.5M	9.82	prox 5; ✓
`incline_polytope (mj)`	2048	1.33M	65536	22.1M	16.68	**no**
`painleve_box_sdf`	2048	99.4k	16384	1.74M	17.56	prox 3, red; ✓
`painleve_box_sdf (mj)`	4096	426k	8192	783k	1.84	✓

scene	B	steps/s	B	steps/s	ratio	changed, check
painleve_box_polytope	2048	474k	16384	5.89M	12.42	prox 5; ✓
painleve_box_polytope (mj)	2048	1.79M	32768	18.6M	10.37	✓
backspin_ball_sdf	2048	109k	16384	1.57M	14.50	tiles; ✓
backspin_ball_sdf (mj)	4096	566k	32768	11.6M	20.52	red, tiles; ✓
backspin_ball_polytope	8192	499k	32768	4.63M	9.29	✓
backspin_ball_polytope (mj)	8192	3.62M	32768	30.6M	8.46	✓
masonry_arch_sdf	128	9.01k	2048	27.5k	3.06	compact; ✓
masonry_arch_sdf (mj)	128	14.8k	2048	65.9k	4.46	red, compact; ✓
masonry_arch_polytope	1024	37.4k	4096	48.4k	1.30	✓
masonry_arch_polytope (mj)	1024	209k	8192	341k	1.64	✓
house_of_cards_sdf	128	2.31k	2048	4.66k	2.02	✓
house_of_cards_sdf (mj)	128	7.18k	2048	17.1k	2.38	✓
house_of_cards_polytope	1024	13.8k	4096	16.3k	1.18	✓
house_of_cards_polytope (mj)	1024	65k	4096	86.4k	1.33	✓
cube_5_sdf	2048	18.7k	2048	110k	5.89	prox 5, red; ✓
cube_5_sdf (mj)	2048	79.5k	2048	278k	3.50	red; ✓
cube_5_polytope	2048	320k	4096	614k	1.92	prox 5; ✓
cube_5_polytope (mj)	2048	1.12M	4096	1.41M	1.26	✓
cube_walled_5_sdf	2048	23.5k	2048	77k	3.28	red, tiles; ✓
cube_walled_5_sdf (mj)	2048	112k	2048	340k	3.04	red; ✓
cube_walled_5_polytope	2048	296k	4096	561k	1.89	prox 5; ✓
cube_walled_5_polytope (mj)	2048	1M	4096	1.25M	1.24	✓
cube_20_sdf	512	6.13k	4096	32.3k	5.27	prox 5, red, tiles; ✓
cube_20_sdf (mj)	512	19.4k	4096	78.6k	4.05	red, tiles; ✓
cube_20_polytope	2048	93k	8192	251k	2.70	prox 3; ✓
cube_20_polytope (mj)	2048	344k	8192	464k	1.35	✓
cube_walled_20_sdf	256	5.21k	4096	17.6k	3.37	red, tiles; ✓
cube_walled_20_sdf (mj)	256	17.8k	4096	66.4k	3.73	red, tiles; ✓
cube_walled_20_polytope	2048	86.7k	4096	217k	2.50	prox 3; ✓
cube_walled_20_polytope (mj)	2048	308k	4096	379k	1.23	✓
arm_5_sdf	2048	80.6k	4096	309k	3.83	✓
arm_5_sdf (mj)	2048	389k	4096	1.34M	3.44	✓
arm_5_polytope	1024	119k	4096	355k	2.97	prox 5; ✓
arm_5_polytope (mj)	1024	329k	4096	739k	2.25	✓
arm_walled_5_sdf	2048	70.1k	4096	203k	2.90	tiles; ✓
arm_walled_5_sdf (mj)	2048	342k	4096	1.09M	3.19	tiles; ✓
arm_walled_5_polytope	1024	87.1k	2048	130k	1.49	✓
arm_walled_5_polytope (mj)	1024	316k	4096	608k	1.92	✓
arm_20_sdf	256	12.4k	4096	118k	9.47	prox 5, tiles; ✓
arm_20_sdf (mj)	256	47.2k	4096	304k	6.44	tiles; ✓
arm_20_polytope	256	15.2k	4096	56k	3.68	✓
arm_20_polytope (mj)	256	30.9k	4096	160k	5.19	**no**
arm_walled_20_sdf	256	11.9k	4096	62.8k	5.27	tiles; ✓
arm_walled_20_sdf (mj)	256	45.8k	4096	295k	6.44	tiles, compact; ✓
arm_walled_20_polytope	256	14.1k	4096	46.3k	3.28	✓
arm_walled_20_polytope (mj)	256	30.4k	4096	146k	4.82	**no**
sharpa_5_sdf	128	5.93k	4096	33.5k	5.66	tiles; ✓

scene	B	steps/s	B	steps/s	ratio	changed, check
sharpa_5_sdf (mj)	128	12k	4096	83.9k	6.98	tiles; ✓
sharpa_5_polytope	512	22.2k	8192	90.9k	4.09	prox 5; ✓
sharpa_5_polytope (mj)	512	46.6k	4096	120k	2.57	✓
sharpa_walled_5_sdf	128	5.8k	4096	31.7k	5.46	tiles; ✓
sharpa_walled_5_sdf (mj)	128	11.4k	4096	82.6k	7.25	tiles; ✓
sharpa_walled_5_polytope	512	22k	4096	100k	4.56	prox 3; ✓
sharpa_walled_5_polytope (mj)	512	46.2k	4096	118k	2.55	✓
sharpa_20_polytope	128	442	1024	1.76k	3.97	prox 5; ✓
sharpa_20_polytope (mj)	128	1.37k	1024	3.73k	2.72	✓
sharpa_walled_20_polytope	128	439	1024	1.75k	3.98	prox 5; ✓
sharpa_walled_20_polytope (mj)	128	1.35k	1024	3.72k	2.76	✓
shadow_5_sdf	128	5.57k	4096	27.9k	5.01	tiles; ✓
shadow_5_sdf (mj)	128	8.5k	2048	50.8k	5.98	tiles; ✓
shadow_5_polytope	512	21.8k	4096	106k	4.85	prox 3; ✓
shadow_5_polytope (mj)	512	33.3k	4096	88.8k	2.67	✓
shadow_walled_5_sdf	128	5.37k	4096	27.4k	5.11	tiles; ✓
shadow_walled_5_sdf (mj)	128	9.95k	2048	51.4k	5.16	tiles; ✓
shadow_walled_5_polytope	512	21.4k	4096	102k	4.77	prox 3; ✓
shadow_walled_5_polytope (mj)	512	31.7k	4096	86.6k	2.73	✓
shadow_20_polytope	128	420	1024	2.64k	6.29	prox 3; ✓
shadow_20_polytope (mj)	128	1.26k	1024	3.2k	2.53	✓
shadow_walled_20_polytope	128	419	1024	2.72k	6.50	prox 3; ✓
shadow_walled_20_polytope (mj)	128	1.25k	1024	3.12k	2.49	✓
sharpa_armadillo_sdf	1024	63.3k	4096	173k	2.74	✓
sharpa_armadillo_sdf (mj)	1024	287k	4096	916k	3.19	✓
sharpa_armadillo_polytope	1024	170k	4096	491k	2.88	prox 3; ✓
sharpa_armadillo_polytope (mj)	1024	278k	4096	452k	1.63	✓
shadow_grasp_sdf	1024	164k	8192	1.43M	8.67	prox 3; ✓
shadow_grasp_sdf (mj)	1024	601k	8192	1.96M	3.26	✓
shadow_grasp_polytope	1024	200k	8192	1.55M	7.73	prox 3; ✓
shadow_grasp_polytope (mj)	1024	543k	8192	1.84M	3.39	✓
nist_threading_m8	1024	396k	4096	859k	2.17	✓
nist_threading_m8 (mj)	1024	615k	8192	2.34M	3.80	compact; ✓
nist_threading_m8_easy	1024	335k	4096	613k	1.83	✓
nist_threading_m8_easy (mj)	1024	512k	8192	1.52M	2.97	tiles; ✓
nist_nutbolt_m8	1024	54.8k	8192	850k	15.50	tiles; ✓
nist_nutbolt_m8 (mj)	1024	253k	4096	1.26M	4.96	✓
nist_nutbolt_m8_s0.25	1024	52.7k	8192	655k	12.42	prox 3, tiles; ✓
nist_nutbolt_m8_s0.25 (mj)	1024	217k	4096	762k	3.51	tiles; ✓
nist_nutbolt_m8_s0.125	512	39.4k	4096	251k	6.38	prox 5, tiles; ✓
nist_nutbolt_m8_s0.125 (mj)	512	107k	2048	290k	2.71	tiles; ✓
nist_nutbolt_m12	1024	55k	8192	1.2M	21.90	prox 3, tiles; ✓
nist_nutbolt_m12 (mj)	1024	275k	4096	1.3M	4.74	tiles; ✓
nist_nutbolt_m12_s0.35	1024	52k	8192	657k	12.64	prox 3, tiles; ✓
nist_nutbolt_m12_s0.35 (mj)	1024	239k	8192	842k	3.52	tiles; ✓

scene	B	steps/s	B	steps/s	ratio	changed, check
nist_nutbolt_m12_s0.175	512	38.3k	4096	255k	6.65	prox 5, tiles; ✓
nist_nutbolt_m12_s0.175 (mj)	512	92.7k	2048	293k	3.16	tiles; ✓
nist_nutbolt_m16	1024	56.8k	8192	1.42M	25.06	prox 3; ✓
nist_nutbolt_m16 (mj)	1024	348k	8192	2M	5.75	tiles; ✓
nist_nutbolt_m16_s0.4	1024	52.4k	4096	595k	11.35	prox 3, tiles; ✓
nist_nutbolt_m16_s0.4 (mj)	1024	244k	2048	768k	3.14	tiles; ✓
nist_nutbolt_m16_s0.2	512	39.4k	4096	310k	7.86	prox 5, tiles; ✓
nist_nutbolt_m16_s0.2 (mj)	512	103k	2048	391k	3.78	tiles; ✓
nist_driven_m8	1024	58.1k	8192	1.45M	25.03	prox 3, tiles; ✓
nist_driven_m8 (mj)	1024	336k	4096	1.38M	4.13	tiles; ✓
nist_driven_m12	1024	56.4k	8192	1.58M	28.09	prox 3, tiles; ✓
nist_driven_m12 (mj)	1024	337k	4096	1.4M	4.17	tiles; ✓
nist_driven_m16	1024	60.5k	8192	1.77M	29.27	prox 3; ✓
nist_driven_m16 (mj)	1024	258k	4096	1.03M	3.99	tiles; ✓
nist_board	512	23.9k	2048	244k	10.18	prox 3; ✓
nist_board (mj)	512	96.5k	1024	260k	2.69	✓
arm_box_6	1024	123k	4096	244k	1.99	✓
arm_box_6 (mj)	1024	403k	8192	1.04M	2.57	✓
arm_box_20	128	12.1k	4096	59.1k	4.89	✓
arm_box_20 (mj)	128	29.5k	4096	239k	8.12	tiles; ✓
nutbolt_05	1024	44.2k	2048	177k	4.01	✓
nutbolt_05 (mj)	1024	145k	2048	344k	2.37	tiles; ✓
nutbolt_05_trilinear	1024	46.3k	2048	145k	3.14	✓
nutbolt_05_trilinear (mj)	1024	160k	2048	398k	2.48	tiles; ✓
nutbolt_1	1024	47.8k	2048	199k	4.16	✓
nutbolt_1 (mj)	1024	204k	2048	557k	2.73	✓
nutbolt_1_trilinear	1024	47.1k	2048	182k	3.86	✓
nutbolt_1_trilinear (mj)	1024	212k	4096	630k	2.98	✓
sharpa_2_sdf	512	27.6k	2048	200k	7.23	prox 5, tiles; ✓
sharpa_2_sdf (mj)	512	79.6k	8192	396k	4.98	tiles; ✓
sharpa_2_polytope	1024	114k	4096	453k	3.98	prox 3; ✓
sharpa_2_polytope (mj)	1024	261k	8192	562k	2.15	✓
sharpa_walled_2_sdf	512	27.4k	2048	130k	4.72	tiles; ✓
sharpa_walled_2_sdf (mj)	512	74.8k	4096	378k	5.06	tiles; ✓
sharpa_walled_2_polytope	1024	111k	4096	448k	4.04	prox 3; ✓
sharpa_walled_2_polytope (mj)	1024	260k	8192	557k	2.14	✓

What the tuning found.

- **Capacities sized for one world.** The viewer sizes MJWarp's capacities for its single world: 4 096 contacts and 12 288 rows per world for five armadillos as surfaces, every Lodos collision fraction at 1.0. On the 38 scenes whose own capacities fit at the family's batch size, calibrated capacities alone made the exact step 1.02–8.2× faster at that batch size (median 1.9×; 3.4× on the five-armadillo drop, 2.8–3.5× on the nut-bolts, 4.9–8.2× on the NIST nut-bolts and driven nuts): several of MJWarp's kernels launch over `njmax` and `naconmax`. On the other 29 the scene's own capacities did not fit in 16 GiB at the family's batch size, and the baseline ran at the largest batch that did, 128–512 worlds on the heavy scenes and 1 024–2 048 on the rest: that is the rest of their ratios in Table 6.

- **The batch size.** Under the exact mode throughput stopped growing at 2 048–4 096 worlds in 45 of the 67 scenes. It went further on the single bodies of the exactness scenes (the incline, the backspin ball, Painlevé's box), to 8 192–32 768, and to 8 192 on 12 more, the driven nuts and five of the NIST nut-bolts among them; the twenty-hand piles stopped at 1 024. Under mj the incline went on to 65 536.
- **Tile sizes per pair** sped up the collision pipeline on 23 scenes under the exact mode, and on the largest of them most:
  - 32 for every armadillo pair of the twenty-armadillo pile (1.19 to 0.83 ms a step), and 16 in the walled one, against its planes too (1.43 to 0.99 ms);
  - 32 for the twenty cubes' pairs (1.80 to 1.01 ms), and 16 in the walled box, against its planes too (2.15 to 1.39 ms);
  - 16–32 for the minor link pairs of the hand piles as surfaces, 20–36 % off: 1.55 to 1.02 ms for five SHARPA hands, 0.49 to 0.34 ms for two, 1.04 to 0.83 ms for five Shadow hands;
  - 16 for the nut against its bolt and the floor on nine of the NIST nut-bolts and driven nuts, 8–40 % off, the densest nuts most (5.44 to 3.27 ms on the M12 nut whose vertices are 0.175 mm apart), and 32 against the floor alone on a tenth;
  - 16–32 for the bodies against the planes in three scenes (the walled five-armadillo and five-cube piles, the backspin ball), 7–9 % off.

  The default of 8 held for the five-armadillo drop it was chosen on, the nut-bolts and the grasps.
- **The budgets, under the guard.**
  - The guard passed 5 proximal iterations in 42 of the 67 scenes, and 3 in 25. They failed mostly on creep at rest, on these scenes:
    - 8 of the 18 armadillo and cube piles;
    - the hand piles as surfaces but the two open SHARPA hands, and the SHARPA grasp as surfaces;
    - the nut-bolts, the NIST M8 nut-bolt, the arch, the cards and the backspin ball.

    The walled five-armadillo pile crept at 15 mm/s with 5 iterations, against 6.6 mm/s with 10. Five iterations passed on every scene with hulls for hands and on every NIST scene but the M8 nut-bolt; three on the NIST scenes but that one and the three densest nuts, which kept five. On the threading, whose own budget is 2 iterations, both passed and ran slower.
  - Two budgets passed the guard's rollout and then failed the final check, which ran out of memory: 5 iterations on the twenty armadillos as hulls, open and walled. Both scenes kept the solver's own budget.
  - Contact reduction passed the guard in 22 of the 23 scenes it was tried on: every scene with Lodos surfaces but the nut-bolts and the NIST scenes, which ship with it on. On the backspin ball as a surface it moved the ball's path. It was kept where it was faster, on dense contact patches: the cubes as lattice surfaces (3.1× on five cubes, 2.6× in the walled box) and, of the single-body exactness scenes, the incline and Painlevé's box as surfaces.
- **The trilinear field.** S_{c0} took the 0.5 mm nut's collision pipeline from 1.29 to 0.72 ms a step at 1 024 worlds, and the 1 mm nut's from 0.55 to 0.38 ms. The tuned exact steps were slower with it all the same: 145k against 177k world-steps/s on the 0.5 mm nut, and 182k against 199k on the 1 mm nut, at 2 048 worlds. Under mj they were faster with it: 398k against 344k, and 630k against 557k.
- **Under mj** the tuning made the step 1.2–20.5× faster (median 3.2×). With no proximal loop to trim, contact reduction was the only budget: it passed the guard in 21 of the 23 scenes it was tried on, and was kept on six, the cubes, the masonry arch and the backspin ball as surfaces. Four mj files failed the final check: on the incline as a surface the box escapes in 326 of the check's 512 worlds, against 324 untuned, the scene's doing and not the tuning's; the incline as a box overflowed its tuned row capacity in the check's rollouts; and on the twenty armadillos as hulls, open and walled, the check itself ran out of memory, so those two are unchecked.

14 Knobs at a glance

Table 7: Every throughput knob: where it is set, whether it changes the answer, and how to set it.

knob	set in	answer	how to set it
batch size B	`make_sim`	same	`tune.py`'s sweep: largest before throughput flattens, within memory
`capacity_tile`, `capacity2`	`col_ kwargs`	same	`calibrate.py`: 1.5× the peak over a state set
`capacity1`	`col_ kwargs`	same	below 1 compacts the tile filter's launch; `tune.py` keeps it if faster
`tile_size`	`col_ kwargs`	same	`tile_tuner.py`; 8 unless a geometry is small or very large
`super_size`	`col_ kwargs`	same	off; try 4–8 with fine tiles and `capacity1` < 1
`dyn_scan`, `grid_stride`	`col_ kwargs`	same	on
spline type	load time	changes	S_{c2}; S_{c0} where the narrowphase dominates and contacts do not slide far
vertex budget	mesh step 3	changes	the fewest vertices that keep the shape where contacts are made
hull count	mesh step 2	changes	the coarsest decomposition that keeps the contact surfaces
`naconmax`, `nconmax`	`make_sim`	same	`calibrate.py`: contacts, Lodos's among them, and broadphase pairs
`njmax`, `njmax_nnz`	`make_sim`	same	`calibrate.py`: rows, Lodos's contacts' among them
`nccdmax`	`make_sim`	same	`calibrate.py`: the hybrid's EPA and manifold counts
`contact_ reduction`	`make_sim`	changes	on with dense Lodos contact patches, if the guard passes
`prox_iters_per_ qp`	`solver_ kwargs`	changes	10; 5 or 3 if the guard passes
`fused_newton`	`solver_ kwargs`	same	off; on for a model of up to 16 dofs, at a calibrated `njmax`
`groups` and their bins	`solver_ kwargs`	changes	off; ray or cone groups where the scene's own checks pass
`row_space`	`solver_ kwargs`	same	off; on where a world has fewer rows than dofs (ray groups on a pile)
`shared_hessian`	`solver_ kwargs`	same	the default: on from 64 padded dofs where it can be built; time it on another GPU
`mode`	`solver_ kwargs`	changes	`"exact"`; `"mj"`, MuJoCo's soft problem, is faster on most scenes, and its contacts creep
`hybrid`	`make_sim`	changes	the default

15 Limitations

- **The guard is a heuristic.** It holds what it measures: penetration, rest, escapes, paths on the non-chaotic scenes. It can accept a budget that changes a quantity it does not measure, such as a seat's settling time or a grasp's slip under a disturbance. It judges depth by a mean and a 99th percentile over every world and sample, so a budget that goes wrong in a few worlds of hundreds can pass, and it has no orientation check: a nut seated cocked is not seen. A scene's own check, and watching it in the viewer, are the last word.
- **Timing noise** is a few percent between rounds. Hence the 3 % threshold, and decisions near it can go either way between runs.
- **`tune.py` measures the independent layout.** For an MPC-style shared batch, run `calibrate.py --layout shared` on the controller's own state set and control sequences.
- **Lodos's stage counts are batch totals.** Per-world peaks are measured only for the contacts, so a file applied below its batch size leaves the collision fractions at 1.0 rather than sizing them for the smaller batch.
- **Throughput is per GPU.** A file's capacities hold on another GPU, but its batch size and tile sizes were chosen for the device it names.